MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Bîtca FAF-242

Report

Laboratory Work No. 3

Cryptography

Topic: Polyalphabetic Substitution

Checked by:
Zaica Maia, asistent universitar

Chișinău - 2026

1 INTRODUCTION AND AIM OF THE WORK

1.1 Polyalphabetic substitution ciphers

A monoalphabetic cipher replaces every letter of the plaintext with the same ciphertext letter throughout the message. As I saw in the previous laboratory work, this is its weak point: the language's letter frequencies remain visible in the ciphertext, so the key can be found by frequency analysis. A cipher is more secure if the distribution of the letters in the ciphertext is as regular as possible and gives no information to the cryptanalyst [1].

One way to flatten the distribution is to combine high frequencies with low ones. If the letter T is sometimes encrypted as a and sometimes as b, and the letter X is also sometimes encrypted as a and sometimes as b, then the high frequency of T is mixed with the low frequency of X, and a and b get a more moderate frequency [1]. This is the idea of polyalphabetic substitution ciphers: the replacement of a letter is not fixed, but changes along the text, depending on parameters such as the position of the letter or its context. The number of possible keys is also much larger than in the monoalphabetic case [1].

1.2 The Playfair cipher

The Playfair cipher was invented by Charles Wheatstone and described for the first time in a document on 26 March 1854. It bears the name of Baron Lyon Playfair, a friend of Wheatstone, who convinced the British government to adopt it for official use. At the beginning it was rejected by the British Foreign Office, because it was considered too hard to understand. The algorithm was used by the British army in the war against the Boers in South Africa, and modified versions were used by the British in the First World War and by the Australian army in the Second World War [1].

The main idea of Playfair is that it encrypts pairs of letters (digraphs) and not single letters. A letter is therefore replaced in different ways, depending on the letter that stands next to it, and the frequencies of single letters are hidden much better than in a monoalphabetic cipher. The cryptanalyst has to work with digraphs, and there are many more possible digraphs than letters [3]. For this reason Playfair is one of the first algorithms that use the principles of modern block ciphers [1]. From the point of view of modern cryptography, however, it is obsolete: a personal computer can find the key and decrypt the message in a few seconds [1]. Today its value is educational, because it helps to understand classical cryptography without complex mathematics [1], [2], [4].

1.3 Purpose of the work

The purpose of this laboratory work is to implement the Playfair cipher for the Romanian alphabet of 31 letters and to understand how polyalphabetic substitution works. I had to write a program in Python that encrypts and decrypts messages with a key given by the user, to check that the input contains only allowed characters, and to compare Playfair with the monoalphabetic ciphers studied before.

1.4 Task (variant 1)

My variant is 1, so I solved task 3.1 of the laboratory guide [1]. The task is to implement the Playfair algorithm for messages in Romanian, with an alphabet of 31 letters. The program has to accept only the letters A-Z and a-z; if the user enters other characters, the program has to show the correct range of characters. The key must have at least 7 characters. The user chooses the operation (encryption or decryption), enters the key and the message or the ciphertext, and receives the ciphertext or the decrypted message. The last step, adding the spaces to the decrypted message according to its meaning, is done manually [1]. Because the Romanian alphabet has 31 letters, my program accepts, besides the 26 Latin letters, the five letters with diacritics: Ă, Â, Î, Ș and Ț.

2 THE PLAYFAIR ALGORITHM

2.1 Preparing the text

Before encryption the message is prepared in three steps. First, all spaces and punctuation are removed and all letters are written with capitals. Second, the text is divided into pairs of letters. If the two letters of a pair are identical, a filler letter is inserted between them. The filler is a letter that is rare in the language, because double letters are frequent and would help the cryptanalyst, and because the filler also reduces the number of words that are easy to guess [1]. In my program the filler is X, and Q is used instead when the repeated letter is X itself. For example, the word ALEEA is divided as AL EX EA. Third, if the number of letters is odd, one more filler is added at the end, so that the last pair is complete [1].

2.2 Building the key matrix

The matrix is built from the key. The repeated letters of the key are removed, keeping only the first appearance of each letter. The remaining letters of the key are written in the matrix row by row, starting from the top left corner, and then the rest of the alphabet is written in alphabetical order [1]. A longer key makes the ciphertext harder to cryptanalyse [1].

For the English alphabet the matrix is 5×5, and the letters I and J share one cell. The Romanian alphabet has 31 letters, which do not fit in a 5×5 matrix, and a 5×6 matrix has only 30 cells. A 6×6 matrix has 36 cells, which is enough for all 31 letters. The guide says that the free cells can be filled with any symbols [1], so I filled the 5 free cells with asterisks (*). In this way no letter has to be removed or replaced.

2.3 Encryption and decryption rules

Each pair of letters is transformed according to the positions of its letters in the matrix, as shown in Table 2.1. Decryption uses the same rules in the opposite direction. The rectangle rule is the same for both operations, because exchanging the columns of the two letters twice gives the original pair [2].

Table 2.1 - Playfair rules for encryption and decryption

Position of the letters	Encryption	Decryption
Same row	each letter is replaced by the letter to its right; after the last column the row continues from the first column	each letter is replaced by the letter to its left; before the first column the row continues from the last column
Same column	each letter is replaced by the letter below it; after the last row the column continues from the first row	each letter is replaced by the letter above it; before the first row the column continues from the last row
Different row and column (rectangle)	each letter is replaced by the letter from its own row and from the column of the other letter	the same rule as for encryption

Let the pair (a, b) have the positions (ra, ca) and (rb, cb) in the matrix M. If s = +1 for encryption and s = −1 for decryption, the letters (c1, c2) of the result are:

rectangle: c1 = M[ra][cb], c2 = M[rb][ca]

same row r: c1 = M[r][(ca + s) mod 6], c2 = M[r][(cb + s) mod 6]

same column c: c1 = M[(ra + s) mod 6][c], c2 = M[(rb + s) mod 6][c]

2.4 Adaptation for the Romanian alphabet

The alphabet used by the program is shown in Figure 2.1. It has 31 letters, and the letter J is kept as a separate letter.

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z

Figure 2.1 - The Romanian alphabet (31 letters)

The asterisks do not contain any letter, so a shift in a row or in a column must never stop on them. If the next cell is an asterisk, the shift continues to the next cell with a letter. For example, in the matrix from Table 4.1 the cell below Ș is an asterisk, so the letter that follows Ș in the column is N, the first letter of the column. The same skip is applied in the opposite direction at decryption, so the two operations remain inverse to each other.

3 IMPLEMENTATION

The program is written in Python in one file. The algorithm is in the class `PlayfairRomanian`, and the validation of the input and the interaction with the user are in separate functions. The messages of the program are written in Romanian, because the texts that are encrypted are Romanian.

3.1 Class and constants

The class begins with the constants that define the algorithm (Figure 3.1). `ALPHABET` contains the 31 letters of the Romanian alphabet, `ROWS` and `COLS` give the size of the matrix (6×6), `FILLER` is the letter X inserted between identical letters, `ALT_FILLER` is the letter Q used when the repeated letter is X, and `EMPTY` is the asterisk placed in the free cells of the matrix.

The constructor receives the key, prepares it, and raises an error if it has fewer than 7 letters. Then it builds the matrix and a dictionary `position`, which gives for every letter its row and column. In this way the program does not have to search the matrix each time it needs the coordinates of a letter.

```
class PlayfairRomanian:  4 usages  new *
    ALPHABET = 'AĂÂBCDEFGHIÎJKLMNOPQRSSTTUVWXYZ'

    ROWS = 6
    COLS = 6

    FILLER = 'X'
    ALT_FILLER = 'Q'
    EMPTY = '*'

    def __init__(self, key :{split} ):  new *
        key = self.prepare_text(key)

        if len(key) < 7:
            raise ValueError("Cheia trebuie să aibă cel puțin 7 caractere.")

        self.matrix = self.build_matrix(key)

        self.position = {
            letter: (r, c)
            for r, row in enumerate [list[Any]] (self.matrix)
            for c, letter in enumerate(row)
            if letter != self.EMPTY
        }
```

Figure 3.1 - Class definition, constants and constructor

3.2 Text preparation and key matrix

The method `prepare_text` removes all whitespace from the text, converts it to capitals and replaces the letters Ş and Ţ (with cedilla), which many keyboards produce, by Ș and Ț. After that it checks every character and raises an error if a character does not belong to the alphabet (Figure 3.2).

The method `build_matrix` writes the key and the alphabet one after the other and uses `dict.fromkeys` to remove the repeated letters, keeping their order. In this way the letters of the key come first, followed by the remaining letters of the alphabet. The list is completed with asterisks up to 36 cells and is divided into 6 rows. The method `print_matrix` shows the matrix in the console, so that the user can check it.

```
def prepare_text(self, text :{split} ):  3 usages  new *
    text = ''.join(text.split()).upper()
    text = text.replace('Ş', 'Ș').replace('Ţ', 'Ț')
    for char in text:
        if char not in self.ALPHABET:
            raise ValueError(
                f"Caracterul '{char}' nu apartine alfabetului românesc."
            )
    return text


def build_matrix(self, key :{__add__} ):  1 usage  new *
    letters = list(dict.fromkeys(key + self.ALPHABET))
    letters.extend([self.EMPTY] * (
        self.ROWS * self.COLS - len(letters)
    ))
    return [
        letters[i:i + self.COLS]
        for i in range(0, len(letters), self.COLS)
    ]


def print_matrix(self):  2 usages  new *
    line = "—" * (self.COLS * 4 - 1)
    print("\nMatricea Playfair:")
    print(f"┌{line}┐")
    for row in self.matrix:
        print("│ " + " │ ".join(row) + " │")

    print(f"└{line}┘")
```

Figure 3.2 - Text preparation and matrix generation

3.3 Splitting the text into pairs

The method `make_pairs` goes through the prepared text and forms the pairs (Figure 3.3). There are three cases. If the current letter is the last one, it is completed with a filler. If the next letter is identical to

the current one, a filler is placed between them and the second letter is left for the next pair. Otherwise the two letters form a pair as they are. The method `filler_for` returns X, or Q if the letter is already X.

Figure 3.3 - Creation of the pairs

3.4 Transformation of a pair

```
def filler_for(self, letter :{__eq__} ):  2 usages  new *
    return self.ALT_FILLER if letter == self.FILLER else self.FILLER

def make_pairs(self, text :{__len__, __getitem__} ):  1 usage  new *
    pairs = []
    i = 0
    while i < len(text):
        a = text[i]
def shift_position(self, r, c, step, direction :{__eq__} ):  4 usages  new *
    for _ in range(1, 7):
        if direction == 'row':
            c = (c + step) % self.COLS
        else:
            r = (r + step) % self.ROWS
        if self.matrix[r][c] != self.EMPTY:
            return r, c
    raise RuntimeError("Nu există poziție validă în matrice.")

def transform_pair(self, a, b, step):  2 usages  new *
    r1, c1 = self.position[a]
    r2, c2 = self.position[b]
    # Cazul 1: caracterele se află pe aceeași linie
    if r1 == r2:
        nr1, nc1 = self.shift_position( r1, c1, step,  direction: 'row')
        nr2, nc2 = self.shift_position(r2, c2, step,  direction: 'row')
        return (self.matrix[nr1][nc1], self.matrix[nr2][nc2])
    # Cazul 2: caracterele se află pe aceeași coloană
    if c1 == c2:
        nr1, nc1 = self.shift_position(r1, c1, step,  direction: 'column')
        nr2, nc2 = self.shift_position(r2, c2, step,  direction: 'column')
        return (self.matrix[nr1][nc1], self.matrix[nr2][nc2])
    # Cazul 3: caracterele formează un dreptunghi
    return (self.matrix[r1][c2], self.matrix[r2][c1])
```

The method `transform_pair` takes a pair of letters and the direction `step` (+1 for encryption and −1 for decryption), finds the positions of the letters and applies one of the three rules (Figure 3.4). For the same row or the same column, it calls `shift_position`, which moves one cell in the required direction, with wrap-around, and repeats the move if it reaches an asterisk. For the rectangle case, the letters exchange their columns.

Figure 3.4 - Transformation of a pair of letters

3.5 Encryption and decryption methods

The method `encrypt` prepares the text, divides it into pairs, transforms every pair with `step = +1` and joins the results (Figure 3.5). The method `decrypt` prepares the ciphertext and checks that it has an

even number of letters. It takes the letters two by two and transforms every pair with `step = −1`. Fillers are not inserted at decryption, because the ciphertext is already divided into pairs.

```
def encrypt(self, plaintext :{split}): 1 usage  new *
    text = self.prepare_text(plaintext)
    pairs = self.make_pairs(text)
    return ''.join(a + b
        for a, b in (
            self.transform_pair(x, y, +1)
            for x, y in pairs
        )
    )

def decrypt(self, ciphertext :{split}): 1 usage  new *
    text = self.prepare_text(ciphertext)
    if len(text) % 2 != 0:
        raise ValueError(
            "Criptograma trebuie să aibă un număr par de litere."
        )
    pairs = zip [tuple[str, str]] (text[0::2], text[1::2])
    return ''.join(a + b
        for a, b in (
            self.transform_pair(x, y, -1)
            for x, y in pairs
        )
    )
```

Figure 3.5 - Complete encryption and decryption methods

3.6 Input validation

The input is checked before the algorithm starts (Figure 3.6). The function `check_letters` verifies that all characters are letters A-Z, a-z or Ă, Â, Î, Ș, Ț (capital or small).

```
VALID_LETTERS = set [str] (
    PlayfairRomanian.ALPHABET
    + PlayfairRomanian.ALPHABET.lower()
    + 'ȘȚșț'
)
RANGE_HINT = (
    "Sunt permise doar literele A-Z și a-z, "
    "plus Ă, Â, Î, Ș, Ț (mari sau mici)."
)
# Verifică dacă textul conține doar caractere permise
def check_letters(text :{__iter__}):...
def check_key(key :{split}):...
def check_message(text :{split}):...
# Verifică criptograma
def check_ciphertext(text :{split}):...
```

Figure 3.6 - Input validation functions

If not, it returns a message with the wrong character and with the allowed range. `check_key` requires at least 7 characters, `check_message` rejects an empty message and `check_ciphertext` rejects

an empty ciphertext or a ciphertext with an odd number of letters. Each function returns `None` if the value is correct and the text of the error otherwise.

3.7 User interface

The function `ask` shows a question and repeats it until the entered value passes the check (Figure 3.7). The functions `encrypt_mode` and `decrypt_mode` ask for the key, show the generated matrix, ask for the message or for the ciphertext and print the result. After encryption the program reminds the user that the spaces are added manually. After decryption it reminds the user to remove the filler letters X and Q and to add the spaces according to the meaning of the message.

```
# Solicită o valoare până când aceasta este validă
def ask(prompt, check):  4 usages  new *
    while True:
        value = input(prompt).strip()
        error = check(value)
        if error is None:
            return value
        print("Eroare:", error)
# Modul de criptare
def encrypt_mode():  1 usage  new *
    key = ask( prompt: "\nCheia (minim 7 litere): ", check_key)
    playfair = PlayfairRomanian(key)
    playfair.print_matrix()
    message = ask( prompt: "\nMesajul de criptat: ", check_message)
    print("\nCriptograma:", playfair.encrypt(message))
    print("Spațiile se adaugă manual, după sensul mesajului.")

# Modul de decriptare
def decrypt_mode():  1 usage  new *
    key = ask( prompt: "\nCheia (minim 7 litere): ", check_key)
    playfair = PlayfairRomanian(key)
    playfair.print_matrix()
    ciphertext = ask( prompt: "\nCriptograma: ", check_ciphertext)
    print("\nMesajul decriptat:", playfair.decrypt(ciphertext))
    print(
        "Elimină manual literele X sau Q adăugate la criptare,"
    )
    print(
        "restabilește eventualele litere J în loc de I "
        "doar dacă este cazul și adaugă spațiile."
    )
```

Figure 3.7 - Input function and the encryption and decryption modes

The function `main` shows the menu (Figure 3.8).

```
def main():  1 usage  new *
    print("Cifrul Playfair - alfabetul românesc")

    while True:
        print("\n1. Criptare")
        print("2. Decriptare")
        print("0. Iesire")

        choice = input("Alege: ").strip()

        if choice == '1':
            encrypt_mode()

        elif choice == '2':
            decrypt_mode()

        elif choice == '0':
            print("Program încheiat.")
            break

        else:
            print("Alege 1, 2 sau 0.")
```

Figure 3.8 - Main menu of the program

4 EXAMPLE OF USE

4.1 Key matrix

As an example I use the key suspicios. After the repeated letters are removed, the key becomes SUSPICIO. These letters are written first, then the remaining letters of the alphabet in alphabetical order, and the last 5 cells are filled with asterisks. The result is shown in Table 4.1.

Table 4.1 - Key matrix for the key "suspicios"

	0	**1**	**2**	**3**	**4**	**5**
0	S	U	P	I	C	O
1	A	Ă	Â	B	D	E
2	F	G	H	Î	J	K
3	L	M	N	Q	R	Ș
4	T	Ț	V	W	X	Y
5	Z	*	*	*	*	*

4.2 Encryption

The message is "acesta este un mesaj secret". After the spaces are removed and the letters are converted to capitals it becomes ACESTAESTEUNMESAJSECRET. It has 23 letters, so a filler letter X is added at the end to form 12 pairs. The 12 pairs are encrypted in Table 4.2. The ciphertext is DSAOZFAOYAPMȘĂAFFCDOȘDȚY (Figure 4.1).

Table 4.2 - Encryption of the message

No.	**Pair**	**Positions (row, column)**	**Rule**	**Result**
1	AC	(1,0) (0,4)	rectangle	DS
2	ES	(1,5) (0,0)	rectangle	AO
3	TA	(4,0) (1,0)	same column	ZF
4	ES	(1,5) (0,0)	rectangle	AO
5	TE	(4,0) (1,5)	rectangle	YA
6	UN	(0,1) (3,2)	rectangle	PM
7	ME	(3,1) (1,5)	rectangle	ȘĂ
8	SA	(0,0) (1,0)	same column	AF
9	JE	(2,4) (1,5)	rectangle	FC
10	SE	(0,0) (1,5)	rectangle	DO
11	CR	(0,4) (3,4)	same column	ȘD
12	ET	(1,5) (4,0)	rectangle	ȚY

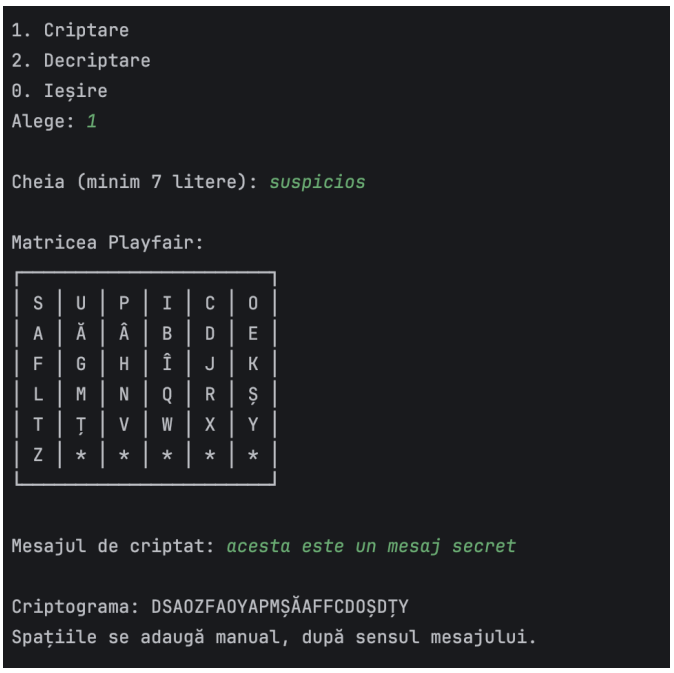


Figure 4.1 - Encryption of the message

4.3 Decryption

For decryption I choose option 2 and enter the same key "suspicios" and the ciphertext DSAOZFAOYAPMȘĂAFFCDOȘDȚY. The program builds the same matrix and transforms the pairs in the opposite direction (Figure 4.2). The result is ACESTAESTEUNMESAJSECRETX. The trailing filler X is removed manually, and spaces are restored according to meaning: "ACESTA ESTE UN MESAJ SECRET".



Figure 4.2 - Decryption with the correct key

4.4 Input validation

Figure 4.3 shows how the program reacts to invalid input. A key with fewer than 7 characters, such as "secret", is rejected with the message: "Eroare: Cheia trebuie să aibă cel puțin 7 caractere." In every case the program prompts the user again until a valid value is provided.

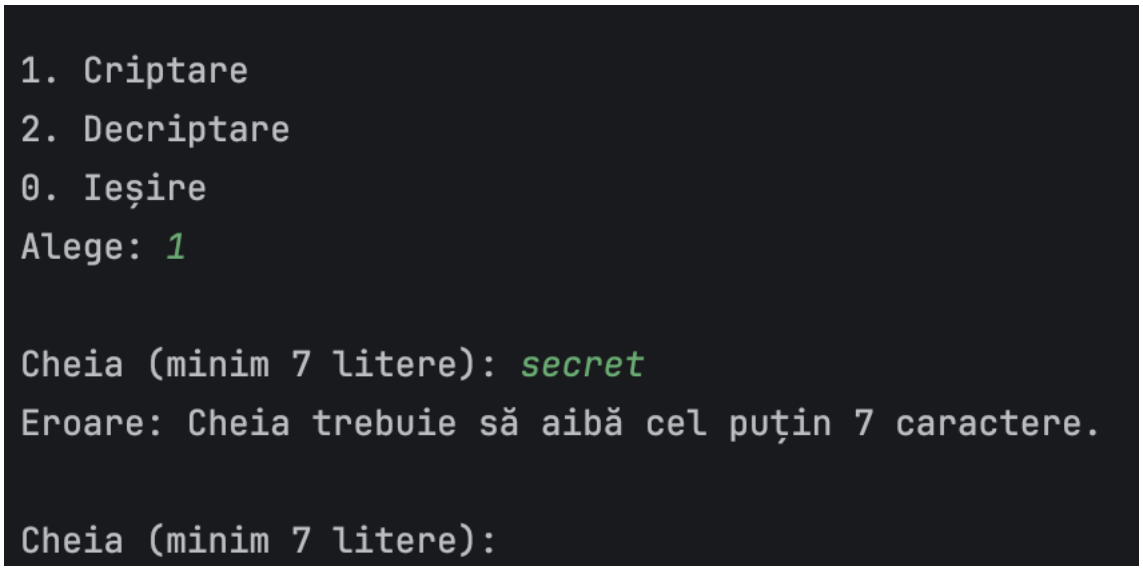


Figure 4.3 - Validation of the input

CONCLUSION

In this laboratory work I implemented the Playfair cipher for the Romanian alphabet of 31 letters. Because 31 letters do not fit in the classical 5×5 matrix, I used a 6×6 matrix, where the 5 free cells are filled with asterisks, and I kept all the letters of the alphabet, including J and the letters with diacritics. The program uses a dictionary with the positions of the letters, a single function for the transformation of a pair, used for both encryption and decryption with the direction as parameter, and a shift that skips the asterisks. In the examples from Section 4 the decrypted text was equal to the original message, except for the filler letters and the spaces, which have to be restored manually.

The work showed me the difference between monoalphabetic and polyalphabetic ciphers. Playfair encrypts pairs of letters, so the frequencies of single letters, which were enough to break the cipher in the previous laboratory work, are hidden. However, the example with the message "sunt foarte miau miau miau" shows that the same pair of letters is always encrypted in the same way, so the frequencies of pairs remain visible. For this reason Playfair is still vulnerable to frequency analysis of digraphs, and a modern computer can break it very fast [1].

The most difficult part was the adaptation of the algorithm to the larger alphabet: the choice of the matrix size, the handling of the empty cells during the shifts and the choice of the filler letters. I also learned to validate the user input and to give clear error messages, and to separate the algorithm from the user interface.

REFERENCES

[1] "Lucrare de laborator nr. 3. Cifruri polialfabetice," laboratory guide, Cryptography course, Technical University of Moldova, FCIM, 2026.

[2] Baeldung, "Playfair Cipher," Baeldung on Computer Science. [Online]. Available: https://www.baeldung.com/cs/playfair-cipher

[3] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Pearson Education.

[4] D. R. Stinson, Cryptography: Theory and Practice, 4th ed. Chapman and Hall/CRC.

[5] Github Repository: https://github.com/gabr1elaa/cs-2026-labs/tree/laboratory-work-3